MSCS-535: Project 3: Project Report: Securing an Online Payment System Against SQL Injection and Cross-Site Scripting - Final Report

---

Sandesh Pokharel

University of the Cumberlands

MSCS-535 Secure Software Development

Brandon Bass

April 13, 2025

1. Introduction

Online payment systems are prime targets for attackers, especially for threats like SQL Injection (SQLi) and Cross-Site Scripting (XSS). These vulnerabilities, if left unchecked, can expose sensitive financial data or compromise user sessions. The goal of this project is to design and implement a secure Spring Boot-based online payment system that defends against these threats while validating user identity prior to transaction processing. This report walks through the implemented pseudocode, real Java code, testing steps, and how the system addresses all the required security mechanisms.

2. System Overview

The application was built using **Java (Spring Boot)**, **Thymeleaf**, and **MySQL**. It offers:

- A frontend payment form
- A Spring controller to handle payment requests
- A service class with business logic
- A JPA-backed repository to securely interact with the database

3. Pseudocode for Security Implementation

3.1 General Pseudocode

```
BEGIN SecureOnlinePaymentSystem

  SHOW form fields: username, cardNumber, cvv, amount
  INPUT username, cardNumber, cvv, amount

  IF any field is empty THEN
     DISPLAY "All fields required"
     STOP
  ENDIF
```

```
 QUERY UserTable WHERE username = input.username
 IF user not found THEN
    DISPLAY "User not registered"
    STOP
 ENDIF

 ESCAPE username, cardNumber, cvv
 PREPARE statement: INSERT INTO Payments(...) VALUES (?, ?, ?, ?)
 BIND inputs and EXECUTE

 DISPLAY "Payment successful" with escaped inputs

END
```

3.2 Our Implemented Pseudocode (Spring Boot + JPA)

```
BEGIN SecurePaymentFlow

 GET /payment → shows form

 POST /payment receives: Payment object
 IF !userRepository.existsByUsername(username) THEN
    RETURN payment-form with error
    STOP
 ENDIF
```

ESCAPE username, cardNumber, cvv with StringEscapeUtils

SET escaped fields in Payment object

SAVE using paymentRepository.save(payment)

DISPLAY receipt in payment-success.html with escaped fields

END

4. Java Implementation

User.java

```
@Entity
@Table(name = "user")
public class User {
   @Id
   private String username;
}
```

Payment.java

```
@Entity
public class Payment {
   @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String username;
    private String cardNumber;
    private String cvv;
    private Double amount;
}
```

UserRepository.java

```
public interface UserRepository extends JpaRepository<User, String> {
    boolean existsByUsername(String username);
}
```

PaymentService.java

```
public boolean processPayment(Payment payment) {
    if (!userRepository.existsByUsername(payment.getUsername())) {
        return false;
    }
    payment.setUsername(StringEscapeUtils.escapeHtml4(payment.getUsername()));
    payment.setCardNumber(StringEscapeUtils.escapeHtml4(payment.getCardNumber()));
    payment.setCvv(StringEscapeUtils.escapeHtml4(payment.getCvv()));
    paymentRepository.save(payment);
    return true;
```

}

PaymentController.java

```
@PostMapping("/payment")
public String submitPayment(@ModelAttribute Payment payment, Model model) {
   boolean success = paymentService.processPayment(payment);
   if (!success) {
      model.addAttribute("payment", payment);
      model.addAttribute("error", "❌ Username does not exist.");
      return "payment-form";
   }
   model.addAttribute("username", payment.getUsername());
   model.addAttribute("amount", payment.getAmount());
   return "payment-success";
}
```

5. SQL Injection Protection

All payment data is stored using Spring Data JPA, which automatically uses **prepared statements** to prevent SQLi attacks. We did not write any raw SQL queries, ensuring complete protection.

6. Cross-Site Scripting (XSS) Protection

Before rendering any user input to the frontend, we used:

StringEscapeUtils.escapeHtml4(input)

7. User Validation

Payment is only allowed if the user exists in the system:

```
if (!userRepository.existsByUsername(payment.getUsername())) {
   return false;
}
```

8. Application Testing

✅ Valid user (alice) → Payment success

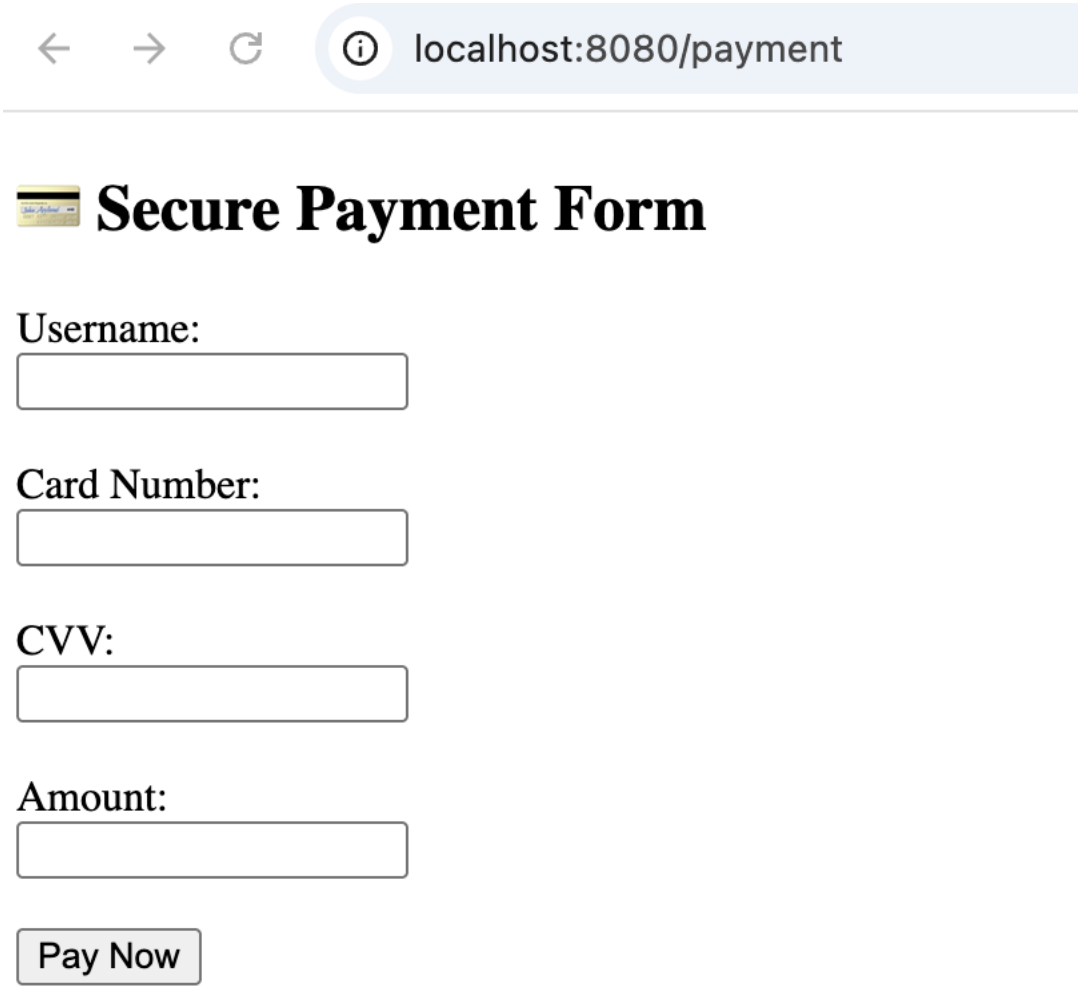
✅ Invalid user → Error message

✅ XSS test input → Escaped safely (<script>alert(1)</script>)

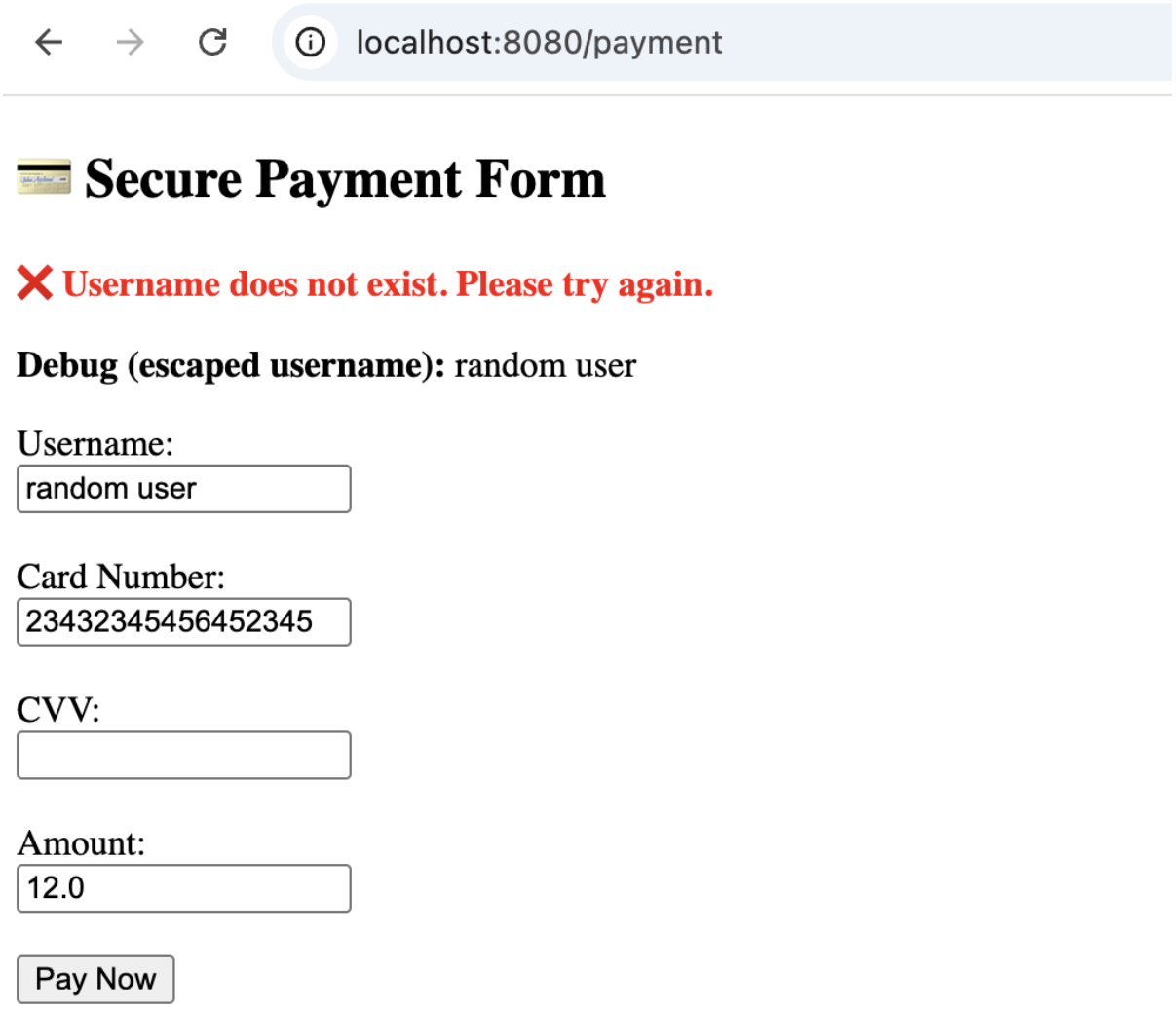
✅ SQLi test → Handled securely (JPA prevents injection)

9. Screenshot Placeholders

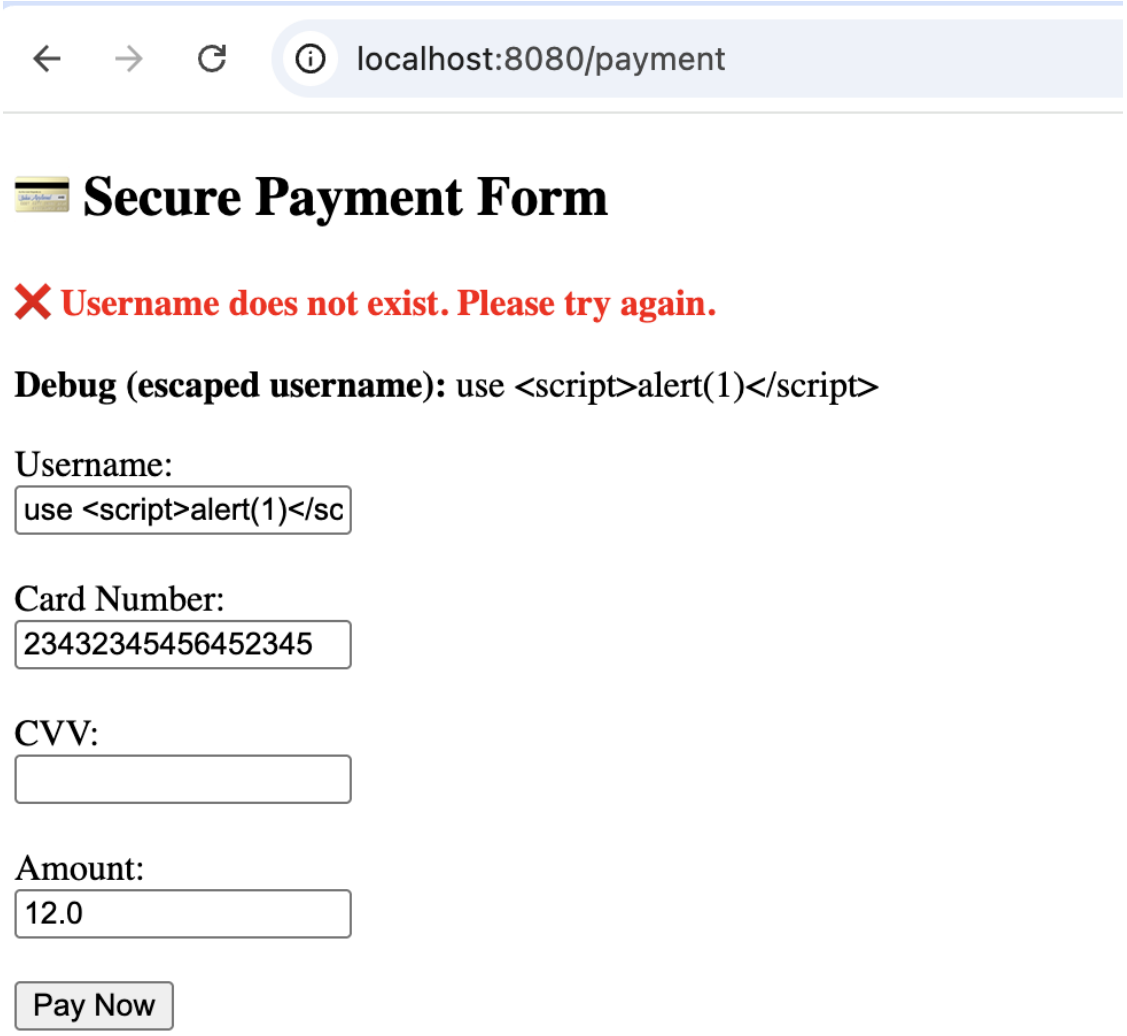
- Screenshot 1: Payment form page loaded at /payment



- Screenshot 2: Form submitted with invalid user (error shown)



- Screenshot 3: Form submitted with <script> username (XSS neutralized in receipt)



10. Conclusion

The project implements a secure online payment system using Spring Boot and MySQL. It successfully addresses SQL Injection and Cross-Site Scripting threats and enforces user validation. With layered protection through JPA and output encoding, the application demonstrates a real-world secure flow suitable for production-grade systems.

11. References

- OWASP. (2021). *Top 10 Web Application Security Risks*. https://owasp.org/Top10/
- NIST. (2020). *Security Considerations for Code Injection*. https://csrc.nist.gov
- Baeldung. (2024). *Preventing SQL Injection in Spring*. https://www.baeldung.com/sql-injection
- Apache Commons Text. (2024). *StringEscapeUtils Documentation*. https://commons.apache.org/proper/commons-text/